

## Crew AI Framework

CrewAI is a lean, **lightning-fast** Python framework built entirely from **scratch**. It empowers developers with both **high-level simplicity** (**easy to use** for developers) and **precise low-level control** (fine-tune the AI when needed, **Adjust** decision-making logic ), ideal for creating autonomous AI agents tailored to any scenario.

### 2 Features:

- 1) **CrewAI Crews:** Crews are designed for autonomous collaboration between multiple AI agents to solve complex tasks.

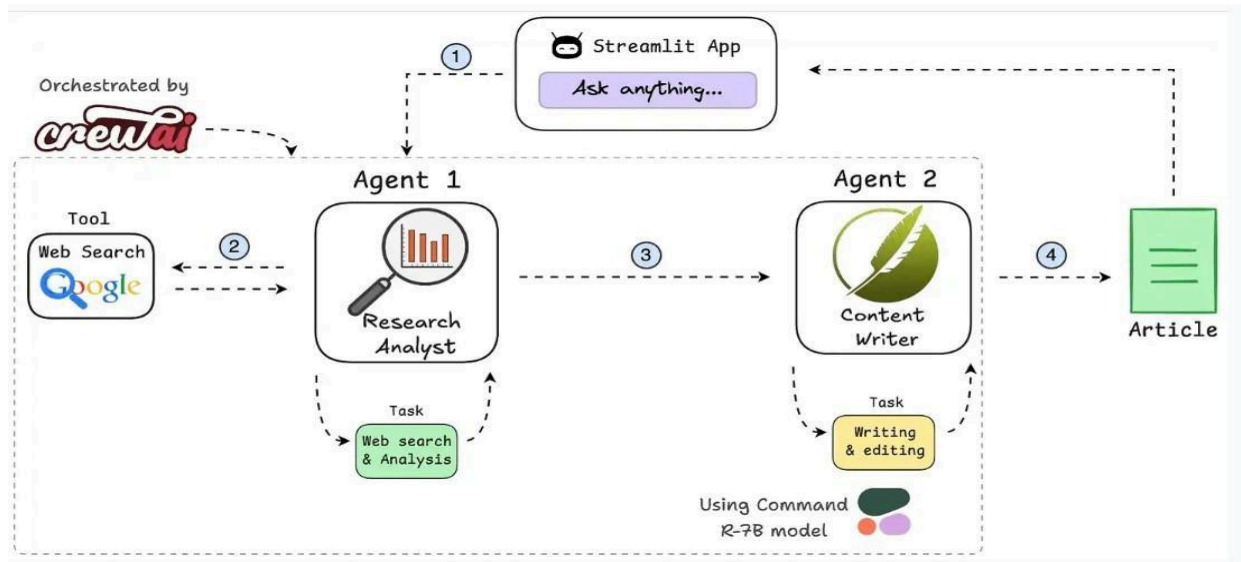
## Real-World Example of a Crew

**Scenario:** Automated Social Media Content Creation and Publishing

A team developed an autonomous system using CrewAI to automate the process of creating and publishing social media content. The workflow involves multiple agents, each with specific roles:

- **Content Scraper Agent:** Utilizes FireCrawl to scrape content from a provided URL, such as a newsletter, and converts it into markdown format.
- **Style Analyzer Agent:** Analyzes the scraped content to understand the author's writing style.
- **Content Generator Agent:** Generates a ready-to-publish draft tailored for platforms like LinkedIn and X (formerly Twitter).
- **Scheduler Agent:** Uses Typefully's API to schedule and publish the generated content on the specified social media channels.

This multi-agent system operates autonomously, handling the entire content creation and publishing process without manual intervention.



2) **CrewAI Flows: Enable granular, event-driven control, single LLM calls** for precise task orchestration and supports Crews natively.

### Real-World Example of a Flow

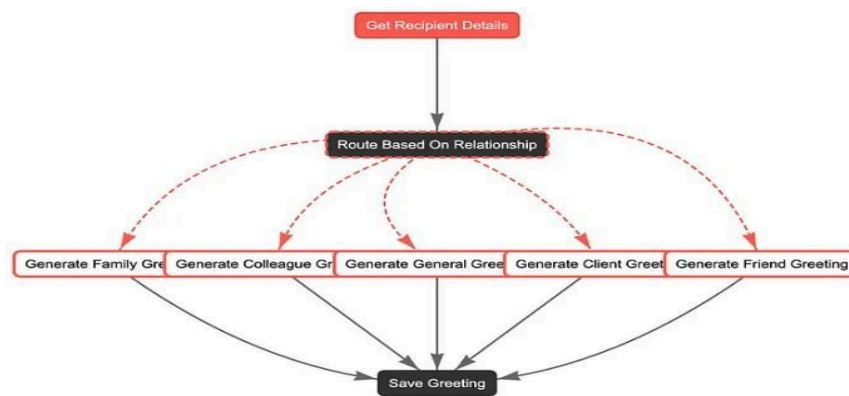
**Scenario:** Personalized Greeting System

A personalized greeting system was built using CrewAI Flows to generate tailored messages based on the recipient's relationship type (e.g., family, friend, colleague, client). The system operates as follows:

- **Greeting Generators:** Multiple agents are designated to generate greetings for different relationship types.
- **Dynamic Routing:** The system dynamically selects the appropriate greeting generator based on the recipient's relationship type using the `@router` decorator.

- **Conditional Execution:** The `or_` and `and_` decorators are used to manage task execution based on the completion of specific agents.
- **Output Handling:** Once the greeting is generated, it's saved to a file (e.g., `greeting.txt`) for further use.

This setup allows for the generation of personalized greetings in an efficient and scalable manner.



3) **Event-Driven:** Trigger tasks based on conditions or previous outputs. Example: "Only send the email after the report is finished."

4) **Single LLM Call Per Step:** Each step is a discrete LLM call for precision. Each task is one clear question to the AI.

Eg: In a code generation application, CrewAI orchestrates a sequence of tasks where each agent performs a specific function. For instance, a **Researcher Agent** might

gather information, followed by a **Writer Agent** that drafts content. Each agent makes a single LLM call to complete its task, ensuring modularity and clarity in the process.

5) **Granular Control:** You decide exactly what happens next. Example: "If the client says YES, do X. If NO, do Y."

Eg: In a personalized greeting system, CrewAI's Flows provide granular control over the execution of tasks. Using decorators like `@start()`, `@listen()`, and `@router()`, developers can define the sequence of operations, handle conditions, and manage state transitions.

Project- Crewai\_basics

 Summary: Agent vs Task

| Feature        | Agent                                 | Task                                  |
|----------------|---------------------------------------|---------------------------------------|
| Role           | Who does the work                     | What needs to be done                 |
| Function       | Executes tasks using tools and memory | Defines an assignment for an agent    |
| Has Tools?     | Yes                                   | No                                    |
| Has Memory?    | Yes (short/long term)                 | No                                    |
| Is Executable? | Indirectly (via tasks)                | Yes (triggered within a process/crew) |
| Used In        | Crews, Processes                      | Processes, Workflows                  |

=====++++++#####=====

Understanding Flows and Crews

CrewAI offers two powerful, complementary approaches that work seamlessly together to build sophisticated AI applications:

**Crews:** Teams of AI agents with true autonomy and agency, working together to accomplish complex tasks through role-based collaboration. Crews enable:

- Natural, autonomous decision-making between agents
- Dynamic task delegation and collaboration
- Specialized roles with defined goals and expertise
- Flexible problem-solving approaches

**Flows:** Production-ready, event-driven workflows that deliver precise control over complex automations. Flows provide:

- Fine-grained control over execution paths for real-world scenarios
- Secure, consistent state management between tasks
- Clean integration of AI agents with production Python code
- Conditional branching for complex business logic

=====

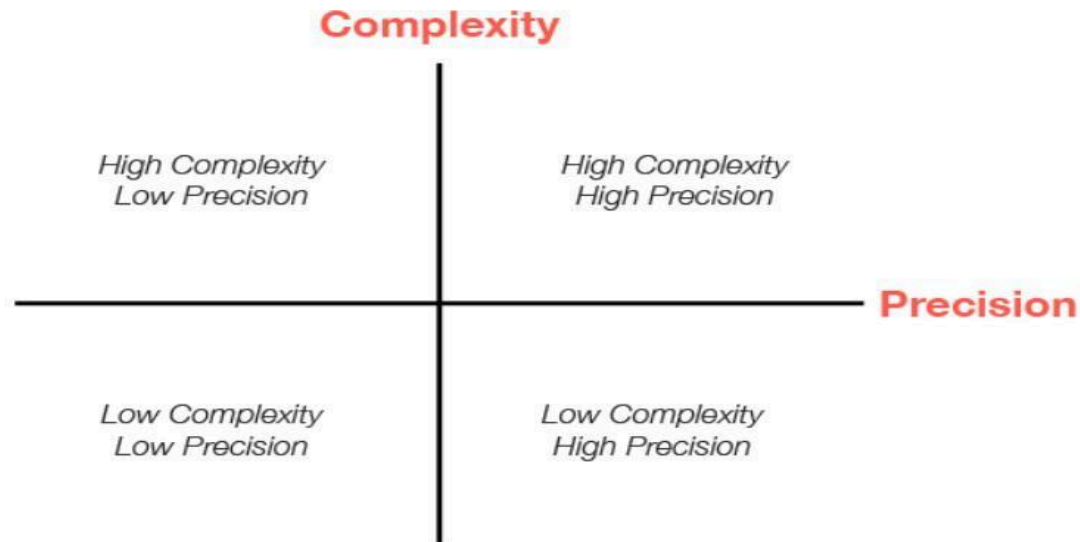
The true power of CrewAI emerges when **combining Crews and Flows**. This synergy allows you to:

- Build complex, production-grade applications
- Balance autonomy with precise control
- Handle sophisticated real-world scenarios
- Maintain clean, maintainable code structure

Learn and Understand- video

### **Strategy- When to use crew, when to use flow, when to use both**

When building AI applications with CrewAI, one of the most important decisions you'll make is choosing the right approach for your specific use case. Should you use a Crew? A Flow? A combination of both?



### What is Complexity?- Quantity

In the context of CrewAI applications, complexity refers to:

- The number of distinct steps or operations required
- The diversity of tasks that need to be performed
- The interdependencies between different components
- The need for conditional logic and branching

### What is Precision?- Quality

Precision in this context refers to:

- The accuracy required in the final output
- The need for structured, predictable results
- The importance of reproducibility
- The level of control needed over each step
- The tolerance for variation in outputs

- 1) **LC,LP**- Recommended Approach: Simple **Crews** with minimal agents
- 2) **HC,LP**- Recommended Approach: Complex **Crews** with multiple specialized agents, its okay if accuracy is a bit less
- 3) **HC,HP**-Recommended Approach: **Flows** orchestrating multiple **Crews** with validation steps.

4) **LC,HP**- Recommended Approach: **Flows** with direct LLM calls or simple **Crews** with structured outputs.

## Project- Trip Planner

### Crewai Agents

**When building effective AI systems, remember this crucial principle:** 80% of your effort should go into designing tasks, and only 20% into defining agents.

**Why? Because even the most perfectly defined agent will fail with poorly designed tasks, but well-designed tasks can elevate even a simple agent. This means:**

- Spend most of your time writing clear task instructions
- Define detailed inputs and expected outputs
- Add examples and context to guide execution
- Dedicate the remaining time to agent role, goal, and backstory

### Core Principles of Effective Agent Design

#### Show agent from vs code

#### 1. The Role-Goal-Backstory Framework

The most powerful agents in CrewAI are built on a strong foundation of three key elements:

#### Role: The Agent's Specialized Function

The role defines **what the agent does and their area of expertise**. When crafting roles:

- 1) Be specific and specialized: **Instead of “Writer,” use “Technical Documentation Specialist”** or “Creative Storyteller”
- 2) Align with real-world professions: Base roles on recognizable professional archetypes
- 3) Include domain expertise: Specify the agent's field of knowledge (e.g., “Financial Analyst specializing in market trends”)

## Goal: The Agent's Purpose and Motivation

The goal directs the agent's efforts and shapes their decision-making process. Effective goals should:

- 1) Be clear and outcome-focused: Define what the agent is trying to achieve
- 2) Emphasize quality standards: Include expectations about the quality of work
- 3) Incorporate success criteria: Help the agent understand what "good" looks like

## Backstory: The Agent's Experience and Perspective

The backstory gives depth to the agent, influencing how they approach problems and interact with others. Good backstories:

- 1) Establish expertise and experience: Explain how the agent gained their skills
- 2) Define working style and values: Describe how the agent approaches their work.
- 3) Create a cohesive persona: Ensure all elements of the backstory align with the role and goal.

## Example: Research Agent

### Before:

**role:** "Researcher"

**goal:** "Find information"

**backstory:** "You are good at finding information online."

### After:

**role:** "Academic Research Specialist in Emerging Technologies"

**goal:** "Discover and synthesize cutting-edge research, identifying key trends,

methodologies, and findings while evaluating the quality and reliability of sources"

**backstory:** "With a background in both computer science and library science, you've mastered the art of digital research. You've worked with research teams at prestigious universities and know how to navigate academic databases, evaluate research quality, and synthesize findings across disciplines. You're methodical in your

approach, always cross-referencing information and tracing claims to primary sources before drawing conclusions."

## Crew: Build your first CrewAI Agent

Code

### Agents

Think of an agent as a specialized team member with specific skills, expertise, and responsibilities. For example, a **Researcher** agent might excel at gathering and analyzing information, while a **Writer** agent might be better at creating content.

### Creating Agents

There are two ways to create agents in CrewAI: using YAML configuration (recommended) or defining them directly in code.

Install Crewai Setup: <https://docs.crewai.com/installation>

**Configure Crew:** In `crew.py`, you instantiate the researcher agent and assign the research task to it. You also define the process flow, specifying whether tasks should be executed sequentially or in parallel.

**Set Up Main Execution:** In `main.py`, you set the topic as an input, initialize the crew, and call the `kickoff()` method to start the workflow.

## Memory in Crewai

CrewAI offers three primary memory configurations to enhance agent capabilities:

### 1. Basic Memory System (Recommended)

This built-in system includes:

- **Short-Term Memory:** Utilizes ChromaDB with Retrieval-Augmented Generation (RAG) to store recent interactions.
- **Long-Term Memory:** Employs SQLite3 to preserve valuable insights and learnings from past executions.
- **Entity Memory:** Captures and organizes information about entities (people, places, concepts) encountered during tasks using RAG.
- Together they form **Contextual Memory**, which maintains coherence across tasks and sessions.

python

```
from crewai import Crew, Agent, Task, Process

crew = Crew(
    agents=[...],
    tasks=[...],
    process=Process.sequential,
    memory=True, # Enables short-term, long-term, and entity memory
    verbose=True
)
```

## 2. User Memory with Mem0 (Legacy Approach)

This approach integrates user-specific memory using Mem0.

## 3. External Memory (New Approach)

Allows integration with standalone external memory providers for specialized requirements.

### Concepts:

#### 1. Quick Start of crewai memory: Enabling Memory

```
from crewai import Crew, Agent, Task, Process

crew = Crew(
    agents=[...],
    tasks=[...],
    process=Process.sequential,
    memory=True, # Enables short-, long-, and entity memory
    verbose=True
)
crew.kickoff()
```

This automatically initializes all three memory types-sort, long, entity using sensible defaults.

## 2. 🗄️ Where Memory Is Stored

CrewAI uses platform-specific directories to store memory and knowledge files, determined via the **appdirs** package- means it stores the memory and knowledge files by default in our system without specifying the path to store the files.

```
C:\Users\{username}\AppData\Local\CrewAI\{project_name}\
├─ knowledge\
├─ short_term_memory\
├─ long_term_memory\
├─ entities\
└─ long_term_memory_storage.db
```

### How to save the memory files in a custom way?

> Override default directory- for short memory:

```
import os
os.environ["CREWAI_STORAGE_DIR"] = "./my_project_storage"

crew = Crew(agents=[...], tasks=[...], memory=True)
```

> Custom storage with SQLite- for long term memory:

```
from crewai.memory import LongTermMemory
from crewai.memory.storage.ltm_sqlite_storage import LTMSQLiteStorage
import os

os.makedirs("./storage", exist_ok=True)

crew = Crew(
    memory=True,
    long_term_memory=LongTermMemory(
        storage=LTMSQLiteStorage(db_path="./storage/memory.db")
    )
)
```

### 3. 🛠 Inspecting & Customizing Storage

Check storage path and contents:

```
from crewai.utilities.paths import db_storage_path
import os

storage = db_storage_path()
print(storage)
print(os.listdir(storage))
```

### 4. Embedding Providers

CrewAI supports various embedding providers for memory storage- OpenAI, Azure, Google AI, Vertex AI, Ollama, Cohere, VoyageAI.

```
# Custom embedder example (Ollama Local)
crew = Crew(
    agents=[agent],
    tasks=[task],
    memory=True,
    embedder={
        "provider": "ollama",
        "config": {
            "model": "mxbai-embed-large",
            "url": "http://localhost:11434/api/embeddings"
        }
    }
)
```

### 5. 💭 When to Add Extra Memory Layers

- For chatbots needing full conversation history or multi-user isolation, CrewAI's built-in memory isn't enough.
- The community suggests combining:
  - conversation history store (e.g., last 30 messages), and
  - long-term memory via external providers (like Mem0 or Zep).

## 6. 🐛 Debugging & Maintenance

### > Resetting Memory Layers (Clean Slate)

#### Why?

Memory layers (short, long, entity, knowledge) can sometimes get into a bad state. Resetting helps start fresh—like clearing browser cookies when a website misbehaves.

#### Run this in CLI

How to do it:

```
bash

crewai reset-memories --short      # clears session data
crewai reset-memories --long      # clears saved insights
crewai reset-memories --entities  # clears tracked entities
crewai reset-memories --knowledge # clears ChromaDB collections
crewai reset-memories --all       # does all of the above
```

### > Inspecting Storage Folders

#### Why?

Ensure memory is actually being written—like checking your downloads folder after downloading a file. Here you're checking if the system is storing memory as expected—like verifying that files exist and directories aren't empty.

```
from crewai.utilities.paths import db_storage_path
import os

base = db_storage_path()
print("Contents:", os.listdir(base))
```

You'll see folders like `knowledge/`, `entities/`, and the SQLite DB file. If they're missing or empty, that's a red flag.

## > Fixing Permission & OS Path Issues

### Why?

Sometimes a folder is read-only or named incorrectly. The system needs write access. Read-only folders mean CrewAI can't save anything (memory is lost). Write access is crucial for creating, updating, or deleting memory files and databases.

```
p = db_storage_path()
print("Writable?", os.access(p, os.W_OK))
```

If it's **False**, change permissions or pick a better path. On Windows, avoid illegal characters.

## > Resolving SQLite “Database Locked” Errors

### Why?

If two processes try to write to the database simultaneously, SQLite refuses and says “database is locked.”

SQLite allows only one write operation at a time. If one process is writing and another tries, you'll get “database is locked”.

This happens when:

- You run multiple CrewAI instances at once.
- Another tool is accessing the same SQLite DB concurrently.

### How to prevent it:

```
import fcntl, os
lock = open(os.path.join(db_storage_path(), ".crewai.lock"), "w")
fcntl.flock(lock, fcntl.LOCK_EX | fcntl.LOCK_NB)
```

This creates a lock file telling other processes to wait. Remember to only run one CrewAI instance at a time.

## > Understanding & Inspecting "Collections" in ChromaDB

### Note:

**Collection wrt memory:** a collection is a folder related to embeddings, documents, and their metadata—like named drawers in a filing cabinet- **collection is a folder which consist of all the memory files and knowledge files.**

Each memory type (like “short-term”) is stored in its own collection.

### How to inspect them:

```
python

import chromadb
from crewai.utilities.paths import db_storage_path

client = chromadb.PersistentClient(path=db_storage_path() + "/knowledge")
print("Collections:", [c.name for c in client.list_collections()])
```

You should see collections like `short_term`, `entity_memory`, etc. If not, something is off.

## > Debug Memory Saving & Retrieval

### Why?

Sometimes memory isn't saved or is never fetched. It's like writing a note but not reading it later.

### How to check:

Search logs for `.save()` calls:

```
Memory saved to long_term_storage.db
```

## 2. User Memory with Mem0 (Legacy Approach)

User Memory refers to the feature in CrewAI that persistently stores and retrieves user-specific information—like past preferences or personal details—across sessions. This data is used to personalize future agent interactions, enhancing consistency and context retention. It uses **Mem0** to store **user-specific memory**.

### Prerequisites

#### 1. Install the Mem0 client:

```
bash

pip install mem0ai
```

#### 2. Set your API key (or optionally org/project IDs):

```
python

import os
os.environ["MEM0_API_KEY"] = "m0-your-api-key"
```

### What is Mem0, and how does its storage work?

Mem0 (“mem-zero”) is an intelligent memory layer aimed at enhancing AI systems with personalized, context-rich memory, boasting improved accuracy and performance over vanilla solutions.

### Where is memory stored? - its types

- 1) In Cloud Mem0, memory lives in Mem0’s backend—no folders appear in your project.
- 2) In Local Mem0, you configure vector databases (e.g. Qdrant, Chroma, SQLite) which store memory data in your project’s file system or specified paths—not directly in Mem0 client code.

## What's the difference between Cloud Mem0 and Local Mem0 ?

| Feature           | ☁ Cloud Mem0                     | 💻 Local Mem0                                                                                                                                 |
|-------------------|----------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------|
| Storage           | Hosted remotely                  | Uses self-hosted DBs (Qdrant, Chroma, SQLite)                                                                                                |
| Setup convenience | Zero infra maintenance           | Requires installing and managing vector DB + configs                                                                                         |
| Configuration     | Minimal code (just user_id)      | Requires config for vector_store, llm, embedder<br><a href="#">dev.to</a> <a href="#">graphlit.com +1</a> <a href="#">docs.crewai.com +5</a> |
| Features          | Full Mem0 platform capabilities  | Locally-limited; you must manage DB health                                                                                                   |
| Latency & tokens  | RPC latency, but managed scaling | Low-latency, run on local infra                                                                                                              |

## Explain user\_id, org\_id, and project\_id

**user\_id:** A unique identifier that represents a specific user within Mem0.

**org\_id (optional):** A higher-level grouping identifier—you can use it to organize multiple users under a “team” or organization in Mem0's cloud system. Use it if you have larger setups or businesses, to segment memory across teams or workspaces.

**project\_id:** A sub-identifier within an org; it helps segment memory by project or specific use-case.

## Common Issues & Workarounds

### A) 🐛 `_user_memory` is None

CrewAI versions (e.g., v0.114.0) had a bug: if `user_memory` is missing, `UserMemory` isn't initialized.

Workaround: Always include `"user_memory": {}` in `memory_config` to ensure initialization.

### B) Local config ignored

Local configs are sometimes overridden or misapplied due to a bug

Tip: Verify that `local_mem0_config` is being passed. Check logs or crew internals to ensure local DB is used.

## Troubleshooting Tips

### 1. Memory shows no entries?

- Ensure conversation contains declarative info (e.g. "I like coffee").
- Confirm user\_id is consistent.

### 2. HTTP 400 from Mem0?

- Means wrong or missing IDs/API key. Ensure user\_id and "user\_memory": {} exist

### 3. Local DB issues?

- Confirm Qdrant (or chosen store) is running and connected.
- Check storage directory paths and permissions.

4. **Reset memory:** use `crew.reset_memories(...)` with types like 'short', 'long', 'entity' to purge caches.

## 3. External Memory

External Memory is a standalone memory system that operates independently from the crew's built-in memory. It's designed for advanced use cases—specialized providers, cross-application memory sharing, and high-scale or custom storage setups. We can connect it to external db's like mysql, mongodb etc.

### Basic Setup with Mem0 Provider

```

import os
from crewai import Agent, Crew, Process, Task
from crewai.memory.external.external_memory import ExternalMemory

os.environ["MEM0_API_KEY"] = "your-api-key"

external_memory = ExternalMemory(
    embedder_config={
        "provider": "mem0",
        "config": {"user_id": "U-123"}
    }
)

crew = Crew(
    agents=[...],
    tasks=[...],
    external_memory=external_memory, # Use instead of memory=True
    process=Process.sequential,
    verbose=True
)

```

## Custom Storage Implementation

```

from crewai.memory.external.external_memory import ExternalMemory
from crewai.memory.storage.interface import Storage

class CustomStorage(Storage):
    def __init__(self):
        self.memories = []

    def save(self, value, metadata=None, agent=None):
        self.memories.append({"value": value, "metadata": metadata, "agent": agent})

    def search(self, query, limit=10, score_threshold=0.5):
        return [m for m in self.memories if query.lower() in str(m["value"]).lower()]

    def reset(self):
        self.memories = []

external_memory = ExternalMemory(storage=CustomStorage())

crew = Crew(
    agents=[...],
    tasks=[...],
    external_memory=external_memory
)

```



**Project- crew\_advanced, memory, knowledge**

**Tasks**

**Task Guardrails**

Code

**Br-----**

**Crewai advanced**

Code

**Knowledge**

3 types of Memory

## **Agent Monitoring & Observability**

OPIK

AgentOps

